

Docker Co

6 bài thực hành nối tiếp — mỗi bài xây trên kiến thức bài trước, từ chạy container đầu tiên đến điều phối multi-container & tối ưu image. 6 progressive hands-on labs, from your first container to multi-container orchestration.

● 2 Beginner

● 2 Intermediate

● 2 Advanced

🕒 ~90 phút

Tác giả: Yên Nguyễn · 2024 · All Rights Reserved



Chuẩn bị môi trường / Prerequisites

Cần cài đặt

- ✓ **Docker Desktop** hoặc Docker Engine (Linux)
- ✓ Trình soạn thảo (VS Code...)
- ✓ Terminal cơ bản

Kiểm tra cài đặt

```
$ docker --version
$ docker compose version
```

BEGINNER

Lab 1 — Chạy container đầu tiên

Run your first container · 🕒 ~10 phút

**Mục tiêu:** Kéo image từ Docker Hub và chạy — hiểu `docker run`, `ps`, `logs`, `stop`.

1

Kiểm tra Docker hoạt động:

```
$ docker run hello-world
```

2

Chạy Nginx ở chế độ nền (`-d`), map cổng 8080:

```
$ docker run -d -p 8080:80 --name myweb nginx
```

3

Xem, đọc log, rồi dừng & xóa:

```
$ docker ps
$ docker logs myweb
$ docker stop myweb && docker rm myweb
```

**Kết quả:** `http://localhost:8080` hiện trang chào Nginx; sau bước 3 container myweb biến mất khỏi `docker ps`.**Thử thách:** Chạy 2 container Nginx cùng lúc trên 2 cổng khác nhau (8080, 8081) không trùng tên.

🎯 **Mục tiêu:** Gắn thư mục máy host vào container bằng **volume** để phục vụ HTML của riêng bạn.

1 Tạo thư mục & file HTML:

```
$ mkdir site
$ echo "<h1>Xin chào Docker!</h1>" > site/index.html
```

2 Chạy Nginx, mount thư mục `site` :

```
$ docker run -d -p 8080:80 -v $(pwd)/site:/usr/share/nginx/html nginx
```

3 Sửa `index.html` rồi reload — thay đổi hiện ngay, **không cần build lại**.

✅ **Kết quả:** `localhost:8080` hiển thị "Xin chào Docker!"; thay đổi file host phản ánh tức thì.

🏆 **Thử thách:** Vì sao dữ liệu ghi vào volume không mất khi container bị xóa? Giải thích ngắn.

🎯 **Mục tiêu:** Tự viết **Dockerfile**, build thành image và chạy container từ image của chính bạn.

📄 server.js

```
const http = require('http');
http.createServer((req,res)⇒{
  res.end('Hello from my image!');
}).listen(3000);
```

📄 Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY server.js .
EXPOSE 3000CMD ["node","server.js"]
```

1 Build image, đặt tên & tag:

```
$ docker build -t my-node-app:1.0 .
```

2 Chạy container từ image vừa build & xem danh sách:

```
$ docker run -d -p 3000:3000 my-node-app:1.0
$ docker images
```

✅ **Kết quả:** localhost:3000 trả về "Hello from my image!"; my-node-app:1.0 có trong docker images .

🏆 **Thử thách:** Chạy docker history my-node-app:1.0 — đếm số layer, đối chiếu số lệnh trong Dockerfile.

🎯 **Mục tiêu:** Dùng **Docker Compose** chạy 2 service (web + Redis) giao tiếp qua Docker network.

📄 docker-compose.yml

```
services:
  web:
    image: nginx:latest
    ports: ["8080:80"]
    depends_on: [cache]
  cache:
    image: redis:alpine
```

1 Khởi động stack & xem service:

```
$ docker compose up -d$ docker compose ps
```

2 Kiểm tra Redis qua tên service `cache`, rồi tắt stack:

```
$ docker compose exec cache redis-cli ping
$ docker compose down
```

✅ **Kết quả:** `compose ps` hiện 2 service "running"; `redis-cli ping` trả về `PONG`.

🏆 **Thử thách:** Thêm service thứ 3 (vd `adminer`) phụ thuộc vào `cache` bằng `depends_on`.

ADVANCED

Lab 5 — Multi-stage build tối ưu image

Shrink your image with multi-stage builds · 🕒 ~20 phút

🎯 **Mục tiêu:** Dùng **multi-stage build** tách bước build khỏi runtime → image nhỏ hơn nhiều lần, tận dụng layer caching.

📄 Dockerfile (multi-stage)

```
# Stage 1 — buildFROM node:18 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2 — runtime (chỉ lấy kết quả build)FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
```

1 Build và xem dung lượng image:

```
$ docker build -t app:slim .
$ docker images app
```

2 Đổi một dòng ở code rồi build lại — quan sát Docker **dùng lại cache** các layer chưa đổi (nhanh hơn nhiều).



Kết quả: Image runtime chỉ còn vài chục MB (nền Alpine) thay vì hàng trăm MB; lần build 2 hiện **CACHED** ở bước không đổi.



Thử thách: Thêm `.dockerignore` loại `node_modules` và giải thích vì sao thứ tự lệnh `COPY` ảnh hưởng cache.

- 🎯 **Mục tiêu:** Đóng gói app 3 tầng — build image API riêng, named volume cho DB, chia network public/private.

📄 docker-compose.yml

```
services:
  frontend:
    image: nginx:alpine
    ports: ["8080:80"]
    networks: [frontend]
  api:
    build: ./api          # build từ Dockerfile riêng
    environment: { DB_HOST: db }
    networks: [frontend, backend]
    depends_on: [db]
  db:
    image: postgres:16-alpine
    environment: { POSTGRES_PASSWORD: secret }
    volumes: [db-data:/var/lib/postgresql/data]
    networks: [backend]   # DB không lộ ra ngoài
volumes: { db-data: {} }
networks: { frontend: {}, backend: {} }
```

- 1 Build & chạy toàn bộ; xem log gộp:

```
$ docker compose up --build -d $ docker compose logs -f
```

- 2 Kiểm chứng cách ly network:

```
$ docker compose exec api ping -c1 db    # OK
$ docker compose exec frontend ping -c1 db # fail
```

- 3 Xóa & tạo lại stack, dữ liệu DB **vẫn còn** nhờ named volume:

```
$ docker compose down
$ docker compose up -d # dữ liệu cũ vẫn nguyên
```

✅ **Kết quả:** 3 service chạy; API nối được DB; frontend bị chặn khỏi DB; dữ liệu tồn tại qua các lần down/up.

🏆 **Thử thách:** Thêm `healthcheck` cho `db` và cho `api` chờ DB "healthy" mới khởi động.



Cheat sheet · Lệnh hay dùng

Image

```
docker build -t name:tag . - build
docker images - liệt kê
docker pull nginx - tải
docker rmi name - xóa
```

Container

```
docker run -d -p 80:80 img
docker ps -a - mọi container
docker logs -f id - log
docker exec -it id sh - shell
```

Compose

```
docker compose up -d
docker compose ps
docker compose logs -f
docker compose down
```

Dọn dẹp / Debug

```
docker system prune
docker inspect id
docker stats
docker volume ls
```



Hoàn thành 6 bài lab!

Bạn đã đi từ container đầu tiên đến full-stack app nhiều tầng. Bước tiếp theo: đẩy image lên Docker Hub và tìm hiểu Kubernetes.